



SCHOOL OF COMPUTING AND INFORMATICS.

DEPARTEMENT OF INFORMATION TECHNOLOGY.

COURSE: ADVANCED DATABASES

Course Code :ITce 2024

GROUP ASSESSMENT OF:

ECOMERCE PLATFOR SYSTEM

Section : 'A' Group 2

GROUP MEMBERS

NO	NAME	ID NUMBER
1	AFRANO KINATO	MTUUR/8919/17
2	AMANUEL DERIGISA	MTUUR/8970/17
3	BAYE ASNAKE	MTUUR/8832/17
4	BEWKET ABIE	MTUUR/7822/17
5	BEZAWIT BETROS	MTUUR/7776/17
6	HUSSEN OMERE	MTUUR/7787/17
7	ADANE DIGA	MTUUR/8415/17
8	BELAY TESHOME	MTUUR/8606/17
9	BINIYAM BIYDIGIE	MTUUR/7866/17
10	ETHIOPIA WORKU	MTUUR/8567/17
11	HAMDU IDRIS	MTUUR/8293/17

Submitted to: Ms. Birhanu

Submission date: 10/09/2018E.C

table content

I. Introduction:	Error! Bookmark not defined.
1.1 Background of the Study:	Error! Bookmark not defined.
1.2 Problem Statement:	Error! Bookmark not defined.
1.3 Objectives of the Project:	Error! Bookmark not defined.
1.4 Scope of the System:	Error! Bookmark not defined.
1.5 System Requirements:	Error! Bookmark not defined.
➤ Feasibility Study	Error! Bookmark not defined.
➤ Functional Requirements	Error! Bookmark not defined.
➤ Non-Functional Requirements	Error! Bookmark not defined.
2. ER Diagram Explanation:	Error! Bookmark not defined.
➤ SQL DDL Scripts	Error! Bookmark not defined.
❖ SQL DDL Scripts:	Error! Bookmark not defined.
3. normalization Form (NF):	Error! Bookmark not defined.
4. Optimized SQL Queries	Error! Bookmark not defined.
5. Transactions & Concurrency	Error! Bookmark not defined.
Concurrent Transaction Scenario	Error! Bookmark not defined.
❖ Concurrency Problems	Error! Bookmark not defined.
6.Security & access control:	Error! Bookmark not defined.
Password Hashing:	Error! Bookmark not defined.
Fraud Detection with Audit Logs	Error! Bookmark not defined.
7.Distributed Database Design	Error! Bookmark not defined.
8.Failure Recovery:	Error! Bookmark not defined.
9.conculation:	37
10.Reference	37

I. Introduction:

Electronic commerce (E-Commerce) has become one of the most important sectors in the modern digital economy. Businesses around the world are increasingly using online platforms to sell products and services because they provide faster communication, wider market access, and better customer convenience. In Ethiopia, the rapid growth of internet access, smartphones, mobile banking, and digital payment systems such as Telebirr has encouraged many businesses to move toward online commerce platforms.

Modern E-Commerce systems require powerful and reliable database systems capable of handling large amounts of data and thousands of simultaneous users. These systems must support product management, customer registration, online ordering, secure payment processing, inventory tracking, logistics coordination, and reporting functionalities. As the number of users and transactions increases, database performance, security, scalability, and reliability become critical challenges.

This project focuses on designing and implementing an advanced database system for a scalable E-Commerce platform operating across multiple cities in Ethiopia such as Addis Ababa, Adama, and Hawassa. The proposed system will use advanced database concepts including normalization, transaction management, concurrency control, indexing, distributed databases, security mechanisms, and failure recovery strategies.

The main purpose of this project is to create a secure, efficient, and scalable database solution that can support modern E-Commerce activities while maintaining data consistency, preventing overselling, and ensuring fast query performance.

1.1 Background of the Study:

The development of information technology has transformed traditional business operations into digital business environments. E-Commerce platforms allow customers to buy products online without visiting physical stores. Customers can browse products, compare prices, make payments, and track deliveries using internet-based systems.

In Ethiopia, the adoption of digital financial technologies such as Telebirr, mobile banking, and online payment services has increased significantly in recent years. This digital transformation has created opportunities for businesses to establish online shopping platforms. However, many organizations still face challenges related to database management, system scalability, and transaction security.

Traditional database systems are often unable to efficiently manage:

1. Large numbers of concurrent users
2. Real-time inventory updates
3. Distributed business operations
4. Secure online transactions
5. Fast product searching and filtering
6. High traffic during promotional sales

Advanced database systems solve these challenges through techniques such as normalization, indexing, concurrency control, distributed database management, replication, backup strategies, and recovery mechanisms.

This project applies advanced database technologies to develop a scalable E-Commerce database system that can support the growing digital economy of Ethiopia while ensuring security, consistency, and high performance.

1.2 Problem Statement:

Modern E-Commerce platforms handle large volumes of customer data, product information, and financial transactions every day. Without a properly designed database system, organizations may experience several serious problems that affect system performance and customer trust.

Some of the major challenges faced by E-Commerce platforms include:

1. Overselling products due to concurrent purchases.
2. Maintaining real-time inventory consistency across multiple users.
3. Managing secure payment transactions.
4. Handling high traffic during promotions and holidays.
5. Preventing unauthorized access and fraud.
6. Managing distributed operations across multiple cities.
7. Recovering data after system failures or crashes.
8. Optimizing query performance for product searches and reports.

For example, when multiple customers attempt to purchase the same limited-stock product simultaneously, poor transaction management may cause negative inventory

values or duplicate orders. Similarly, weak security systems may expose sensitive customer information such as payment details and passwords.

Therefore, there is a need for an advanced database system that provides:

- Strong transaction management
- High concurrency support
- Secure payment processing
- Distributed database operations
- Efficient query optimization
- Reliable backup and recovery mechanisms

This project addresses these challenges by designing a scalable and secure E-Commerce database system suitable for Ethiopian business environments.

1.3 Objectives of the Project:

➤ General Objective

To design and implement a scalable, secure, and distributed advanced database system for a modern E-Commerce platform operating across multiple cities in Ethiopia.

➤ Specific Objectives

The specific objectives of the project are:

- To design a fully normalized relational database schema.
- To implement secure payment processing mechanisms.
- To support concurrent transaction processing for multiple users.
- To prevent overselling of products during simultaneous purchases.
- To optimize product search and reporting queries.
- To implement role-based access control and security mechanisms.
- To maintain audit logs for monitoring user activities and transactions.
- To design a distributed database architecture across multiple regions.
- To implement backup, checkpointing, and failure recovery strategies.
- To ensure ACID compliance for transaction processing.
- To improve database scalability and system performance.

1.4 Scope of the System:

The proposed E-Commerce database system focuses on supporting online shopping operations across multiple cities in Ethiopia. The system includes

database functionalities related to customers, products, orders, payments, inventory, and logistics management.

The scope of the system includes:

➤ **Customer Management**

- Customer registration
- Login and authentication
- Customer profile management

➤ **Product Management**

- Product registration
- Product categorization
- Product search and filtering
- Product inventory tracking

➤ **Order Management**

- Shopping cart management
- Order placement
- Order tracking
- Order status updates

➤ **Payment Processing**

- Telebirr payments
- Bank transfer payments
- Cash on delivery payments

➤ **Inventory Management**

- Real-time stock updates
- Low-stock monitoring
- Inventory synchronization

➤ **Security Features**

- Password hashing
- Data encryption
- Role-based access control
- Audit logging

➤ **Distributed Database Operations**

- Regional data fragmentation
- Replication strategies
- Distributed transaction management

➤ **Failure Recovery**

- Backup and restore mechanisms
- Write-Ahead Logging (WAL)
- Check pointing and crash recovery

The system will primarily operate across:

- AddisAbaba
- Adama
- Hawassa

1.5 System Requirements:

The implementation of the E-Commerce platform requires both hardware and software resources. The recommended hardware includes Intel Core i5 processors, 8 GB RAM, SSD storage, and stable internet connectivity.

The software requirements include MySQL or PostgreSQL database systems, Windows or Linux operating systems, and database administration tools such as MySQL Workbench or pg Admin.

The database system must support advanced features such as ACID transactions, replication, concurrency control, indexing, backup and recovery, and role-based access control.

➤ **Feasibility Study**

Technical Feasibility:

The project is technically feasible because modern relational database systems support transactions, replication, indexing, and security mechanisms.

Economic Feasibility:

The system uses open-source technologies, reducing implementation costs while improving business efficiency.

Operational Feasibility:

The proposed system improves customer service, order processing, inventory management, and reporting capabilities.

➤ **Functional Requirements**

The system shall:

- Allow customers to register and login securely.
- Allow product browsing and searching.
- Support online ordering and payments.
- Automatically update inventory after purchases.
- Generate sales reports.
- Maintain audit logs.
- Track shipment and delivery status.
- Support concurrent users efficiently.

➤ **Non-Functional Requirements**

The system must provide:

- High performance
- Scalability
- Reliability
- Security
- Availability
- Data consistency
- Fault tolerance
- Fast query response time

2.ER Diagram Explanation:

The Entity Relationship (ER) Diagram visually represents the database structure and relationships between entities.

➤ **Customers Entity**

Stores customer information including:

- Customer ID
- Full name
- Email
- Phone number
- City

Relationship:

- One customer can place multiple orders.

➤ **Categories Entity**

Stores product categories such as:

- Electronics
- Fashion
- Home Appliances

Relationship:

- One category contains multiple products.

➤ **Products Entity**

Stores product details including:

- Product name
- Price
- Description
- Category ID

Relationship:

- Each product belongs to one category.

➤ **Inventory Entity**

Stores stock quantity information for products.

Relationship:

- One inventory record belongs to one product.

➤ **Orders Entity**

Stores customer order details including:

- Order date
- Total amount
- Status

Relationship:

- One customer can have many orders.

➤ **Order_Items Entity**

Stores products included in each order.

Relationship:

- One order contains many order items.
- One product may appear in many orders.

➤ **Payments Entity**

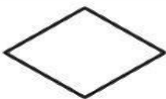
Stores payment information including:

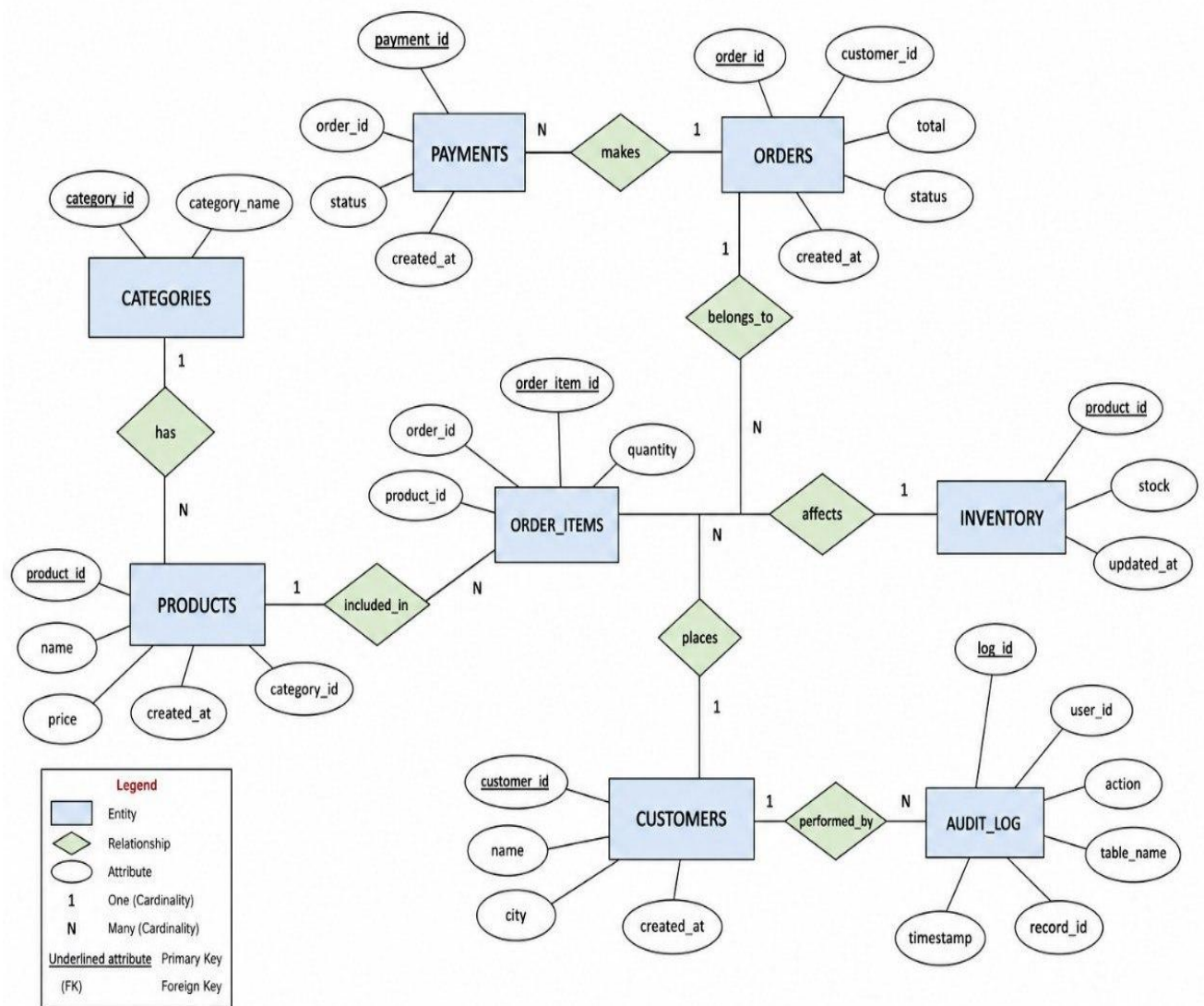
- Payment method
- Payment status
- Transaction date

➤ **Audit_Log Entity**

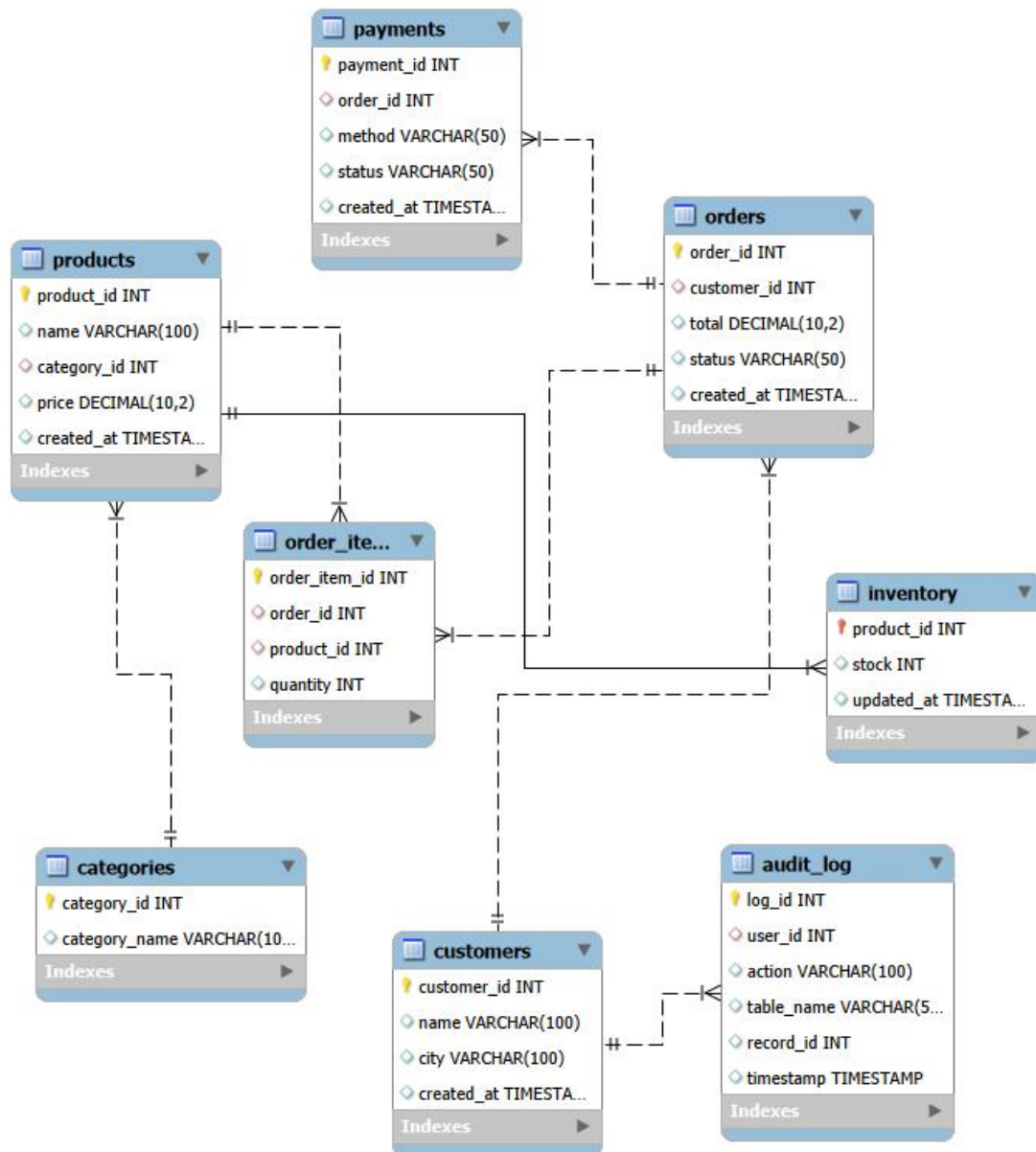
Stores records of user activities and system transactions.

❖ **Notation ER-Diagram for E-commerce Platform system**

Symbols	Meaning
	Entity
	Relationship
	Attribute
	Key Attribute
	Cardinality Ratio N: 1 for E1: E2 in R



2.1 Relational schema:



2.2 IMPLEMENTATION OF SQL QUERIES:

➤ SQL DDL Scripts

```
CREATE DATABASE E_COMMERCE;
USE E_COMMERCE;
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    city VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
Desc customers;
```

Field	Type	Null	Key	Default	
customer_id	int	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
city	varchar(100)	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
➤ CREATE TABLE Categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(100)
);
Desc categories;
```

Field	Type	Null	Key	Default	Extra
category_id	int	NO	PRI	NULL	
category_name	varchar(100)	YES		NULL	

```
➤ CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100),
    category_id INT,
    price DECIMAL(10,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);
Desc products;
```

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
name	varchar(100)	YES		NULL	

	00)				
category_id	int	YES	MUL	NULL	
price	decimal(10,2)	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
➤ CREATE TABLE Inventory (
  product_id INT PRIMARY KEY,
  stock INT,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
Desc inventory;
```

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
stock	int	YES		NULL	
updated_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
➤ CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  total DECIMAL(10,2),
  status VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
Desc orders;
```

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_id	int	YES	MUL	NULL	
total	decimal(10,2)	YES		NULL	
status	varchar(50)	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
➤ CREATE TABLE Order_Items (
  order_item_id INT PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
```

```

    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
Desc order_items;

```

Field	Type	Null	Key	Default	Extra
order_item_id	int	NO	PRI	NULL	
order_id	int	YES	MUL	NULL	
product_id	int	YES	MUL	NULL	
quantity	int	YES		NULL	

```

➤ CREATE TABLE Payments (
    payment_id INT PRIMARY KEY,
    order_id INT,
    method VARCHAR(50),
    status VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
Desc payments;

```

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	
order_id	int	YES	MUL	NULL	
method	varchar(50)	YES		NULL	
status	varchar(50)	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```

➤ CREATE TABLE Audit_Log (
    log_id INT PRIMARY KEY,
    user_id INT,
    action VARCHAR(100),
    table_name VARCHAR(50),
    record_id INT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Customers(customer_id)
);

```

```
desc Audit_Log;
```

Field	Type	Null	Key	Default	Extra
log_id	int	NO	PRI	NULL	
user_id	int	YES	MUL	NULL	

action	varchar(100)	YES	NULL	
table_name	varchar(50)	YES	NULL	
record_id	int	YES	NULL	
timestamp	timestamp	YES	CURRENT_TIMESTAMP	DEFAULT_GENERATED

❖ SQL DDL Scripts:

```
USE E_COMMERCE;
```

```
➤ INSERT INTO Categories (category_id, category_name) VALUES
(1, 'Electronics'),
(2, 'Fashion');
SELECT *FROM catagories;
```

```
category_id  category_name
```

```
1 Electronics
```

```
2 Fashion
```

```
➤ INSERT INTO Customers (customer_id, name, city) VALUES
(1, 'Biniyam', 'Addis Ababa'),
(2, 'Baye', 'Adama'),
(3, 'Hamidu', 'Hawassa');
```

```
SELECT *FROM CUSTOMERS;
```

```
customer_id  name      city      created_at
1 Biniyam   Addis Ababa 4/18/2026 13:07
2 Baye      Adama      4/18/2026 13:07
3 Hamidu    Hawassa    4/18/2026 13:07
```

```
➤ INSERT INTO Products (product_id, name, category_id, price) VALUES
(1, 'Laptop', 1, 15000),
(2, 'Phone', 1, 8000),
(3, 'Shoes', 2, 1200);
Select *from products;
```

```
product_id  name      category_id  price      created_at
```

1	Laptop	1	15000	4/18/2026 13:07
2	Phone	1	8000	4/18/2026 13:07
3	Shoes	2	1200	4/18/2026 13:07

➤ INSERT INTO Inventory (product_id, stock) VALUES

(1, 10),

(2, 20),

(3, 50);

Select *from inventory;

product_id	stock	updated_at
1	10	4/18/2026 13:07
2	20	4/18/2026 13:07
3	50	4/18/2026 13:07

➤ INSERT INTO Orders (order_id, customer_id, total, status) VALUES

(1, 1, 8000, 'Delivered'),

(2, 2, 15000, 'Pending');

Select *from orders;

order_id	customer_id	total	status	created_at
1	1	8000	Delivered	4/18/2026 13:07
2	2	15000	Pending	4/18/2026 13:07

➤ INSERT INTO Order_Items (order_item_id, order_id, product_id, quantity) VALUES

(1, 1, 2, 1),

(2, 2, 1, 1);

Select *from order_items;

order_item_id	order_id	product_id	quantity
1	1	2	1
2	2	1	1

➤ INSERT INTO Payments (payment_id, order_id, method, status) VALUES

(1, 1, 'Telebirr', 'Completed'),

(2, 2, 'Cash on Delivery', 'Pending');

Select *from payments;

payment_id	order_id	method	status	created_at
1	1	Telebirr	Completed	4/18/2026 13:07
2	2	Cash on Delivery	Pending	4/18/2026 13:07

```
➤ INSERT INTO Audit_Log (log_id, user_id, action, table_name, record_id)
VALUES
(1, 1, 'Placed Order', 'Orders', 1),
(2, 1, 'Payment Completed', 'Payments', 1),
(3, 2, 'Created Order', 'Orders', 2);
```

```
Select * from Audit_Log;
```

Log_id	User_id	action	Table_Name	record_id
1	1	Placed Order	orders	1
2	1	Payment completed	patments	1
3	2	Created order	orders	2

3.normalization Form (NF):

❖ Unnormalized Form:

Definition

A table is in Unnormalized Form (UNF) when it contains repeating groups, multi-valued attributes, or no defined primary key. There are no rules enforced — data is stored as a single flat structure.

UNF Violations Present

- No primary key defined on the table
- Repeating groups: a single order (OrderID=3) appears in multiple rows because the customer bought multiple products
- Customer data (name, city) is repeated in every row — redundancy
- Product data (price, category) is embedded inside the order row — no separation
- Multi-valued data mixed in a single flat record

UNF Table: Orders_Flat

(Single flat table — all data mixed together, no keys, repeating rows for multi-item orders)

OrderID	CustomerName	CustomerCity	ProductName	ProductPrice	CategoryName	Quantity
1	Biniyam	Addis Ababa	Phone	8000	Electronics	1
2	Baye	Adama	Laptop	15000	Electronics	1
3	Hamidu	Hawassa	Shoes	1200	Fashion	2
3	Hamidu	Hawassa	Phone	8000	Electronics	1

Notice: Customer Hamidu (rows 3 & 4) appears twice because Order 3 has two items. This is a classic repeating-group violation.

❖ First Normal Form (1NF):

Definition

A table is in First Normal Form (1NF) when:

- Each column contains only atomic (indivisible) values
- Each column contains values of the same type
- Each row is uniquely identifiable (primary key exists)
- There are no repeating groups or arrays

➤ Changes Made from UNF → 1NF

- Added a composite primary key: (OrderID, OrderItemID) to uniquely identify each row
- Gave each order item its own row — eliminating the repeating group (Order 3 with two products now becomes two separate rows with their own OrderItemID)
- All values are now atomic — no cell contains a list or multiple values
- Column types are consistent across all rows

Remaining Problems After 1NF

1NF solves atomicity but does NOT solve:

- Partial dependencies — CustomerName and City depend only on OrderID, not on the full composite key (OrderID + OrderItemID)
- ProductName and Price depend only on ProductID, not the full key
- Data redundancy is still significant — customer data is repeated per row

1NF Table: Orders_1NF

(Each row is atomic and uniquely identified by OrderID + OrderItemID)

OrderID	OrderItemID	CustomerName	City	ProductID	ProductName	Price	Quantity
1	1	Biniyam	Addis Ababa	2	Phone	8000	1
2	2	Baye	Adama	1	Laptop	15000	1
3	3	Hamidu	Hawassa	3	Shoes	1200	2
3	4	Hamidu	Hawassa	2	Phone	8000	1

Primary Key: (OrderID, OrderItemID) | All cells atomic | No repeating groups

❖ Second Normal Form (2NF):

Definition

A table is in Second Normal Form (2NF) when:

- It is already in 1NF
- Every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies)

➤ Changes Made from 1NF → 2NF

- Created separate Customers table — CustomerName and City depend only on CustomerID
- Created separate Products table — ProductName, Price, CategoryID depend only on ProductID
- Created Orders table — only order-level data (CustomerID FK, Total, Status, Timestamp)
- Created Order_Items table — only item-level data with FK references to Orders and Products
- All partial dependencies removed — every attribute now fully depends on its table's PK

2NF Tables

➤ Customers:

CustomerID	Name	City	created_at
1	Biniyam	Addis Ababa	2026-04-18
2	Baye	Adama	2026-04-18
3	Hamidu	Hawassa	2026-04-18

➤ **Products:**

ProductID	Name	CategoryID	Price	created_at
1	Laptop	1	15000	2026-04-18
2	Phone	1	8000	2026-04-18
3	Shoes	2	1200	2026-04-18

➤ **Orders:**

OrderID	CustomerID	Total	Status	created_at
1	1	8000	Delivered	2026-04-18
2	2	15000	Pending	2026-04-18

➤ **Order_Items:**

OrderItemID	OrderID	ProductID	Price	Qty
1	1	2	8000	1
2	2	1	15000	1

Primary Keys: CustomerID, ProductID, OrderID, OrderItemID (each table has its own single-column PK). All foreign keys established.

❖ **Third Normal Form (3NF):**

Definition

A table is in Third Normal Form (3NF) when:

- It is already in 2NF
- There are no transitive dependencies — a non-key attribute must not depend on another non-key attribute

In other words: non-key → non-key dependencies must be removed.

➤ **Changes Made from 2NF → 3NF:**

- Extracted Categories table — CategoryName depends on CategoryID, not ProductID (removes transitive dependency in Products)
- Products table now only stores category_id FK, not category_name directly
- Extracted Inventory table — Stock quantity is a separate business concern with its own update lifecycle
- Extracted Payments table — payment method and status are independent of order attributes
- Extracted Audit_Log table — logging is an independent cross-cutting concern

- All transitive dependencies eliminated — every non-key attribute now depends only on its table's PK

❖ 3NF Tables (Final Schema):

➤ Categories (new — extracted from Products):

CategoryID	CategoryName
1	Electronics
2	Fashion

➤ Products (updated — CategoryName replaced by CategoryID FK):

ProductID	Name	CategoryID	Price	created_at
1	Laptop	1	15000	2026-04-18
2	Phone	1	8000	2026-04-18
3	Shoes	2	1200	2026-04-18

➤ Customers (unchanged from 2NF):

CustomerID	Name	City	created_at
1	Biniyam	Addis Ababa	2026-04-18
2	Baye	Adama	2026-04-18
3	Hamidu	Hawassa	2026-04-18

➤ Orders (unchanged from 2NF):

OrderID	CustomerID	Total	Status	created_at
1	1	8000	Delivered	2026-04-18
2	2	15000	Pending	2026-04-18

➤ Order_Items (unchanged from 2NF):

OrderItemID	OrderID	ProductID	Price	Qty
1	1	2	8000	1
2	2	1	15000	1

➤ Inventory (new — extracted from Products):

ProductID	Stock	updated_at
-----------	-------	------------

1	10	2026-04-18
2	20	2026-04-18
3	50	2026-04-18

➤ **Payments (new — extracted from Orders):**

PaymentID	OrderID	Method	Status	created_at
1	1	Telebirr	Completed	2026-04-18
2	2	Cash on Delivery	Pending	2026-04-18

➤ **Audit_Log (new — independent entity):**

1	1	Placed Order	Orders	1	2026-04-18
2	1	Payment Completed	Payments	1	2026-04-18
3	2	Created Order	Orders	2	2026-04-18

4.Optimized SQL Queries

➤ **Product Search (by category, price range, keyword):**

```
SELECT product_id, name, price, stock_quantity
FROM Products
WHERE category_name = 'Electronics'
AND price BETWEEN 5000 AND 20000
AND name LIKE '%Laptop%'
ORDER BY price ASC;
```

➤ **Top-Selling Products**

```
SELECT p.product_id, p.name, SUM(oi.quantity) AS total_sold
FROM Products p
JOIN Order_Items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY total_sold DESC
LIMIT 5;
```

➤ **Customer Order History:**

```

SELECT o.order_id, o.total_amount, o.status, o.created_at, p.method
FROM Orders o
LEFT JOIN Payments p ON o.order_id = p.order_id
WHERE o.customer_id = 1
ORDER BY o.created_at DESC;

```

➤ **Daily/Monthly Revenue Reports:**

```

SELECT CAST(created_at AS DATE) AS order_date, SUM(total_amount) AS
daily_revenue
FROM Orders
WHERE status = 'Delivered'
  AND created_at >= '2026-05-01 00:00:00'
  AND created_at <= '2026-05-31 23:59:59'
GROUP BY CAST(created_at AS DATE)
ORDER BY order_date DESC;

```

➤ **Low-Stock Product Alerts:**

```

SELECT product_id, name, stock_quantity
FROM Products
WHERE stock_quantity < 5;

```

❖ **Explanation:**

1. Index Usage Strategies:

- ✓ To make these queries support high concurrency during Ethiopian holiday promotions, build these indexes:

```

CREATE INDEX idx_products_search ON Products (category_name, price); (Optimizes
Query 1)

```

```

CREATE INDEX idx_orders_customer ON Orders (customer_id, created_at
DESC); (Optimizes Query 3)

```

```

CREATE INDEX idx_orders_revenue ON Orders (status, created_at); (Optimizes Query
4)

```

2. Query Execution Plans (EXPLAIN):

- ✓ **Index Range Scan:** For Query 1 and Query 4, instead of scanning millions of rows across all cities, the database engine performs an *Index Range Scan*. It immediately narrows the search window down to the specified price or date boundaries.
- ✓ **Hash Match / Merge Join:** For Query 2, the execution plan chooses an in-memory *Hash Join* rather than a slow *Nested Loop*. It matches the keys from Products to Order_Items instantly.

3. Core Performance Improvements:

- ✓ **Sargability:** In Query 4, the initial syntax used `WHERE DATE(created_at) = ...`. Wrapping columns in functions forces the database to evaluate every single row manually. Replacing it with explicit timestamp constraints (`>=` and `<=`) makes the query **sargable**, which allows index usage.
- ✓ **No Wildcard Selects:** Avoiding `SELECT *` drastically lowers data payload sizes, optimizing the limited network bandwidth between regional nodes in Addis Ababa, Adama, and Hawassa.

5. Transactions & Concurrency

A transaction is a sequence of database operations treated as one logical unit of work. In an E-commerce platform, transactions are used when customers place orders, make payments, and update inventory. Every transaction must follow the ACID properties:

Every transaction follows ACID properties:

- Atomicity — all steps succeed or none do
- Consistency — database rules always remain valid (stock never goes negative)
- Isolation — concurrent transactions do not interfere with each other
- Durability — committed data is permanently saved even after a crash

Concurrent Transaction Scenario

Product: Laptop (product_id = 1) | Stock: 1 unit remaining

Two customers attempt to buy the last Laptop at the same time:

- Transaction A — Customer Biniyam (Addis Ababa)
- Transaction B — Customer Baye (Adama)

Without any concurrency control:

Transaction A (Biniyam)	Transaction B (Baye)
BEGIN;	
SELECT stock FROM Inventory	
WHERE product_id = 1;	
Returns: stock = 1	
	BEGIN;
	SELECT stock FROM Inventory

	WHERE product_id = 1;
	Returns: stock = 1 ← stale!
UPDATE Inventory	
SET stock = stock - 1	
WHERE product_id = 1;	
COMMIT; -- stock = 0	
	UPDATE Inventory
	SET stock = stock - 1
	WHERE product_id = 1;
	COMMIT; -- stock = -1 ✗ OVERSOLD!

Problem: Both read stock=1 before either writes. Both commit. Stock becomes -1. The Laptop is sold twice.

❖ Concurrency Problems

1. Lost Update

Occurs when two transactions read the same value and both write updates — one update overwrites the other and is permanently lost.

```
UPDATE Inventory
SET stock = stock - 1
WHERE product_id = 1;
COMMIT;
UPDATE Inventory
SET stock = stock - 1
WHERE product_id = 1;
COMMIT;
```

❖ Result:

Stock becomes -1 ✗

Problem:

- Tx B used outdated data.
- Tx A's update was effectively lost.

❖ **Fix:** Use FOR UPDATE or SERIALIZABLE isolation to lock the row.

```
SELECT stock
FROM Inventory
WHERE product_id = 1
FOR UPDATE;
```

2. Dirty Read

Occurs when a transaction reads data that another transaction has modified but not yet committed. If the first transaction rolls back, the second has used invalid data.

```
SELECT stock
FROM Inventory
WHERE product_id = 1;
ROLLBACK;
```

❖ **Result:** Tx B used uncommitted ("dirty") data ✗

Problem: Tx B believed the product was out of stock.

The update was never permanent.

❖ **Fix:** Use READ COMMITTED isolation level.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

3. Non-Repeatable Read

Occurs when a transaction reads the same row twice and gets different results because another transaction committed a change in between.

```
SELECT stock
FROM Inventory
WHERE product_id = 1;
UPDATE Inventory
SET stock = 1
WHERE product_id = 1;
COMMIT;
SELECT stock
FROM Inventory
WHERE product_id = 1;
```

❖ **Result:** Tx A gets different values in the same transaction ✗

Problem:

- Reports and calculations become inconsistent.

❖ **Fix:** Use REPEATABLE READ or SERIALIZABLE isolation level to ensure the same data remains unchanged during the transaction.

1. Isolation Levels

Level	Dirty Read	Non-Repeat. Read	Phantom Read	E-Commerce Use
READ COMMITTED	✓ Prevented	✗ Possible	✗ Possible	Product browsing, reports
SERIALIZABLE	✓ Prevented	✓ Prevented	✓ Prevented	Checkout, payment

➤ **READ COMMITTED — for product browsing:**

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN;
SELECT p.name, p.price, i.stock
FROM Products p
JOIN Inventory i
ON p.product_id = i.product_id
WHERE p.category_id = 1
AND i.stock > 0;
COMMIT;
```

➤ **SERIALIZABLE — for checkout and payment:**

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
-- Lock product stock row
SELECT stock
FROM Inventory
WHERE product_id = 1
FOR UPDATE;
-- Reduce stock safely
UPDATE Inventory
SET stock = stock - 1
WHERE product_id = 1
AND stock > 0;
-- Create order
INSERT INTO Orders (customer_id, total, status)
VALUES (1, 15000, 'Pending');

-- Create payment record
INSERT INTO Payments (order_id, method, status)
VALUES (LAST_INSERT_ID(), 'Telebirr', 'Pending');
COMMIT;
```

2. Locking Mechanisms

FOR UPDATE acquires an exclusive lock on the row. Any other transaction trying to read or write that row must wait until the lock is released.

Transaction A (Biniyam) — WITH LOCK	Transaction B (Baye) — BLOCKED THEN REJECTED
BEGIN;	
SELECT stock FROM Inventory	
WHERE product_id = 1 FOR UPDATE;	
-- Lock acquired. stock = 1.	
	BEGIN;
	SELECT stock FROM Inventory
	WHERE product_id = 1 FOR UPDATE;
	-- ☒ BLOCKED — waiting for A's lock

3. Transactions with COMMIT / ROLLBACK

➤ Successful order (COMMIT):

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
SELECT stock
FROM Inventory
WHERE product_id = 2
FOR UPDATE;
UPDATE Inventory
SET stock = stock - 1
WHERE product_id = 2;
INSERT INTO Orders (order_id, customer_id, total, status)
VALUES (3, 3, 8000, 'Pending');
INSERT INTO Order_Items (order_item_id, order_id, product_id, quantity)
VALUES (3, 3, 2, 1);
INSERT INTO Payments (payment_id, order_id, method, status)
VALUES (3, 3, 'Telebirr', 'Completed');
INSERT INTO Audit_Log (log_id, user_id, action, table_name, record_id)
VALUES (4, 3, 'Placed Order', 'Orders', 3);
COMMIT;
```

➤ Failed payment — ROLLBACK:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
SELECT stock
FROM Inventory
WHERE product_id = 1
FOR UPDATE;
UPDATE Inventory
SET stock = stock - 1
WHERE product_id = 1;
INSERT INTO Orders (order_id, customer_id, total, status)
VALUES (4, 2, 15000, 'Pending');
ROLLBACK;
```

6.Security & access control:

➤ Role-Based Access Control (RBAC)

Role-Based Access Control assigns database permissions based on user roles rather than individual users. This means every user is given a role (Admin, Seller, or Customer), and that role determines exactly what they can read, write, or manage in the database.

Role Overview

Role	Who Uses It	Access Level	Primary Responsibilities
Admin	Database/System Administrator	Full Access	Manage all users, products, orders, reports, and system settings

Role	Who Uses It	Access Level	Primary Responsibilities
Seller	Product vendors / shop owners	Partial Access	Add/update products, manage stock and pricing, view orders
Customer	Registered shoppers	Limited Access	Browse products, place orders, make payments, view own history

SQL Implementation — Creating Roles

The following SQL commands create the three roles in the database:

```

✓ -- Step 1: Create the roles
  CREATE ROLE admin;
  CREATE ROLE seller;
  CREATE ROLE customer;
✓ -- Step 2: Grant Admin full privileges on all tables
  GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin;
  GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin;
✓ -- Step 3: Grant Seller product and inventory management
  GRANT ALL PRIVILEGES ON Products TO seller;
  GRANT ALL PRIVILEGES ON Inventory TO seller;
  GRANT SELECT ON Orders TO seller;
  GRANT SELECT ON Order_Items TO seller;
✓ -- Step 4: Grant Customer limited access
  GRANT SELECT ON Products TO customer;
  GRANT SELECT ON Categories TO customer;
  GRANT SELECT, INSERT ON Orders TO customer;
  GRANT SELECT, INSERT ON Order_Items TO customer;
  GRANT SELECT, INSERT ON Payments TO customer;
✓ -- Step 5: Assign roles to actual database users
  GRANT admin TO db_admin_user;
  GRANT seller TO seller_user;
  GRANT customer TO customer_user;

```

Encryption for Payment info

The platform uses AES-256 encryption (Advanced Encryption Standard) to protect payment details. The database stores only the encrypted version — the original value is never saved in plain text.

```

✓ -- Store encrypted Telebirr payment reference
-- AES_ENCRYPT(value, secret_key) in MySQL
INSERT INTO Payments (payment_id, order_id, method, encrypted_reference, status)
VALUES (1, 1, 'Telebirr', ,AES_ENCRYPT('09-1234-5678-transaction-ref', 'platform_secret_key_256bit'),
'Completed'
);
-- To read the payment reference (only admins can do this)
SELECT payment_id, order_id, method,
AES_DECRYPT(encrypted_reference, 'platform_secret_key_256bit') AS transaction_ref,
status
FROM Payments
WHERE payment_id = 1;

```

Encrypting Customer Phone Numbers

```
✓ -- Customers table with encrypted phone number
CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  name        VARCHAR(100),
  email       VARCHAR(150),
  phone       VARBINARY(255), -- Stores encrypted binary data
  city        VARCHAR(100),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

✓ -- Insert customer with encrypted phone number
INSERT INTO Customers (customer_id, name, email, phone, city)
VALUES (
  1,
  'Biniyam',
  'biniyam@example.com',
  AES_ENCRYPT('0912345678', 'platform_secret_key_256bit'),
  'Addis Ababa'
);
```

Password Hashing:

Unlike encryption (which is reversible), password hashing is a one-way process. This means even if the database is breached, attackers cannot recover the original passwords from the stored hash values.

Account Lockout Policy

To prevent brute-force password attacks, the system locks accounts after 5 consecutive failed login attempts:

```
✓ -- Trigger to lock account after 5 failed login attempts
DELIMITER $$
CREATE TRIGGER after_failed_login
AFTER UPDATE ON User_Accounts
FOR EACH ROW
BEGIN
  IF NEW.failed_attempts >= 5 THEN
    UPDATE User_Accounts
    SET locked_until = DATE_ADD(NOW(), INTERVAL 30 MINUTE)
    WHERE account_id = NEW.account_id;
  END IF;
END$$
DELIMITER ;

✓ -- Check lock status before allowing login
SELECT account_id, username,
CASE
  WHEN locked_until > NOW() THEN 'LOCKED'
  ELSE 'ACTIVE'
```

```

END AS account_status
FROM User_Accounts
WHERE username = 'biniyam_aa';

```

❖ Audit logging

Audit logging creates a permanent, tamper-evident record of important system activities. It answers the questions: who did what, to which record, and when? This is critical for fraud detection, security investigations, and compliance.

Audit Log Table Schema

```

✓ -- Comprehensive Audit_Log table
CREATE TABLE Audit_Log (
    log_id      INT PRIMARY KEY AUTO_INCREMENT,
    user_id     INT NOT NULL,
    username    VARCHAR(100),
    action      VARCHAR(100) NOT NULL,
    table_name  VARCHAR(50),
    record_id   INT,
    old_value   TEXT,          -- Value before the change (for
UPDATE/DELETE)
    new_value   TEXT,          -- Value after the change (for
INSERT/UPDATE)
    ip_address  VARCHAR(45),   -- IP address of the user
    status      ENUM('SUCCESS', 'FAILED') DEFAULT 'SUCCESS',
    timestamp   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES User_Accounts(account_id)
);

```

➤ Login Attempt Logging

```

✓ -- Stored procedure: Log login attempts (success and failure)
DELIMITER $$
CREATE PROCEDURE log_login_attempt(
    IN p_username VARCHAR(100),
    IN p_ip_address VARCHAR(45),
    IN p_success BOOLEAN
)
BEGIN
    DECLARE v_user_id INT DEFAULT NULL;
    DECLARE v_status VARCHAR(10);

    SELECT account_id INTO v_user_id
    FROM User_Accounts WHERE username = p_username;

    SET v_status = IF(p_success, 'SUCCESS', 'FAILED');

    INSERT INTO Audit_Log (user_id, username, action, ip_address, status)
    VALUES (v_user_id, p_username, 'LOGIN_ATTEMPT', p_ip_address, v_status);

✓ -- Increment failed attempts counter
    IF NOT p_success THEN
        UPDATE User_Accounts

```

```

SET failed_attempts = failed_attempts + 1
WHERE username = p_username;
ELSE
UPDATE User_Accounts
SET failed_attempts = 0, last_login = NOW()
WHERE username = p_username;
END IF;
END$$
DELIMITER ;

```

Fraud Detection with Audit Logs

The audit log is not just a record — it is a fraud detection tool. The following query identifies suspicious patterns such as multiple failed logins or unusually large orders placed in quick succession.

Detecting Brute-Force Login Attacks

```

✓ -- Find accounts with more than 3 failed logins in the last hour
SELECT username, ip_address, COUNT(*) AS failed_count
FROM Audit_Log
WHERE action = 'LOGIN_ATTEMPT'
AND status = 'FAILED'
AND timestamp >= NOW() - INTERVAL 1 HOUR
GROUP BY username, ip_address
HAVING COUNT(*) > 3
ORDER BY failed_count DESC;

```

Detecting Unusual Order Activity

```

-- Find customers who placed more than 5 orders in one hour
-- (possible account takeover or bot activity)
SELECT al.user_id, al.username, COUNT(*) AS order_count
FROM Audit_Log al
WHERE al.action = 'ORDER_PLACED'
AND al.timestamp >= NOW() - INTERVAL 1 HOUR
GROUP BY al.user_id, al.username
HAVING COUNT(*) > 5
ORDER BY order_count DESC;

```

7. Distributed Database Design

To handle high user traffic during Ethiopian promotional seasons and ensure seamless operations across multiple business locations, a robust distributed database architecture is essential. The following sections provide the technical blueprint for the platform's distributed design spanning Addis Ababa, Adama, and Hawassa.

➤ Data Fragmentation Strategy

We employ Horizontal Fragmentation (also known as sharding) to distribute table rows across geographical nodes based on location. This optimizes data localization so that a customer in Hawassa reads from the Hawassa node rather than querying the Addis Ababa hub.

➤ **Fragment Configuration Table**

Fragment / Shard	Target Region	Sharding Key Evaluation	Hosting Node
Shard 1 (Central)	Addis Ababa	shipping_city = 'Addis Ababa'	Addis Ababa Main Hub
Shard 2 (East)	Adama	shipping_city = 'Adama'	Adama Edge Node
Shard 3 (South)	Hawassa	shipping_city = 'Hawassa'	Hawassa Edge Node

➤ **Fragmentation Rules**

- **Horizontal Sharding (Orders, Order_Items, Payments):** Transactional tables are partitioned across regional nodes based on the shipping city. A user ordering from Hawassa reads and writes directly to the Hawassa node, avoiding long-distance latency to Addis Ababa.
- **Full Replication (Products, Categories):** Product catalogs are heavily read but infrequently updated. These tables are fully copied to all regional nodes, enabling thousands of local users to browse with sub-millisecond latency during promotions.

➤ **Replication Strategy**

The system uses a hybrid replication topology to balance reliability and operational scale at the same time.

- **Multi-Master (Peer-to-Peer) Model across Hubs:** The three regional nodes (Addis Ababa, Adama, Hawassa) each act as masters for their local transactional shards, while using bidirectional asynchronous replication to propagate global updates — such as inventory adjustments or new product listings — to every other node.
- **Master-Slave (Primary-Replica) Model within Regions:** Within each city's data center, a primary (master) node handles all local writes (order placements), while one or more read-replicas absorb heavy read traffic (product browsing and filtering) during peak hours.

➤ **Data Consistency Approach**

To manage distributed transactions without sacrificing overall system performance, workflows are separated into two distinct consistency categories:

✓ **Strong Consistency — Two-Phase Commit (2PC)**

Applied exclusively to payment authorization and financial settlement workflows.

- When a transaction involves shifting balances or updating a global wallet like Telebirr, a distributed lock manager coordinates two phases:
 - Prepare Phase: all participating nodes confirm they can commit the change.

- Commit Phase: nodes synchronize and apply the change, or roll back entirely if any node fails.
- This guarantees the payment state is perfectly synchronized across all nodes, preventing double charges or phantom payments.

Eventual Consistency — Saga Pattern

Applied to long-running supply chain and order tracking workflows, e.g. placed → shipped → delivered.

- If a network disruption isolates a node, it logs changes locally.
- Message queues (RabbitMQ or Kafka) propagate the updates to other regional nodes once communication is restored.
- This avoids blocking the entire order pipeline when one region temporarily goes offline.

➤ Trade-offs: CAP Theorem Analysis

The CAP Theorem states that a distributed database can guarantee at most two out of three properties simultaneously: Consistency (C), Availability (A), and Partition Tolerance (P). Because real-world networks will always experience packet drops, Partition Tolerance (P) is non-negotiable in any production deployment. The e-commerce platform makes the following deliberate trade-offs:

Workflow	CAP Profile	Trade-off Risk	Justification
Checkout & Payments	CP — Consistency + Partition Tolerance	If the Hawassa-to-Addis Ababa payment link drops, checkout requests on the isolated node will temporarily fail.	Rejecting a checkout is safer than allowing two users in two cities to buy the last high-value item — prevents overselling and financial discrepancy.
Product Catalog & Browsing	AP — Availability + Partition Tolerance	A price update by a seller in Addis Ababa may be visible to Adama customers a few minutes late during network lag.	Thousands of shoppers during a holiday sale must keep browsing smoothly. Minor price-sync delays are acceptable to avoid crashes.

8.Failure Recovery:

The e-commerce platform implements a multi-layered failure recovery strategy to ensure data durability, transaction integrity, and rapid restoration after any hardware crash, power outage, or network failure. This section details the core mechanisms.

➤ Write-Ahead Logging (WAL)

Core Rule	Every transaction change must be written to a non-volatile WAL log file BEFORE modified data pages are written to the permanent database disk.
------------------	--

How it works for the e-commerce platform:

- When a customer places an order via Telebirr or bank transfer, the changes (new order row, inventory decrement, payment record) are instantly written to a sequential WAL log on disk.
- Only after the log write is confirmed does the database apply the changes to the main data pages.
- If a power failure or crash occurs at any moment, no data is silently lost because the log is already durable.
- During restart, the WAL log acts as the authoritative source of truth for what happened.

➤ Checkpointing

Checkpointing is the mechanism that limits how far back in the WAL log the system needs to scan during recovery, shortening restart time after a crash.

Process

- At scheduled intervals (e.g., every 5 minutes during normal operation, every 30 seconds at peak load), the database engine flushes all dirty pages from RAM memory to the permanent disk.
- A special CHECKPOINT record is then appended to the WAL log, marking the exact point at which all prior data is safely on disk.

Benefit During Recovery

- Upon rebooting after a crash, the system only needs to replay WAL log entries created after the last CHECKPOINT record.
- Without checkpointing, recovery would require replaying the entire WAL history from the beginning, which could take hours on a busy e-commerce database.

➤ Backup and Restore Strategies

Strategy	Description	Schedule / Trigger
Full Backups	A complete snapshot of the entire e-commerce database is taken and stored offline.	Daily, during off-peak hours (2:00 AM – 4:00 AM local time).
Incremental Backups	Only the data modifications made since the previous backup are saved, reducing storage space and backup duration.	Every 6 hours, or after any bulk import or mass update operation.
Point-in-Time Recovery (PITR)	WAL log segments are continuously archived to a remote regional storage node (e.g., from Addis Ababa hub to Hawassa backup node), enabling administrators to restore the	Continuous streaming of WAL segments in real time.

Strategy	Description	Schedule / Trigger
	database to any exact microsecond before a corruption event.	

➤ Crash Recovery Scenario — ARIES Protocol

If the database server suddenly crashes during peak operations (e.g., the Addis Ababa hub crashes mid-holiday-sale), it executes the three-phase ARIES recovery algorithm upon reboot:

Phase	What Happens	E-Commerce Example
1. Analysis Phase	Scans the WAL log forward from the last checkpoint. Identifies which transactions had committed before the crash and which ones were still in progress.	The system discovers that a Telebirr payment (Transaction T1) had committed, but a stock-decrement for a concurrent order (Transaction T2) was interrupted mid-write.
2. Redo Phase	Replays all changes from every COMMITTED transaction to guarantee they are fully and correctly saved to disk.	The completed Telebirr payment is re-applied to the Payments table and the corresponding Order record is confirmed with status = Paid.
3. Undo Phase	Reverses all changes from incomplete or uncommitted transactions using the log's before-images, restoring the database to a clean state.	The partial stock decrement from the interrupted T2 is rolled back, restoring the inventory count to its pre-crash value and preventing phantom lost stock.

➤ Data Consistency After Failure

After any recovery operation completes, the platform guarantees the following invariants across all three regional nodes:

- **Accurate Stock Counts:** Inventory values will perfectly reflect only fully committed sales. No accidental overselling can occur because partial inventory decrements are rolled back by the Undo phase.
- **No Broken Transactions:** Every record in the Payments table will have a matching, consistent row in the Order_Items table. Orphaned payment records or unlinked orders cannot exist after recovery.
- **Distributed Realignment:** Any regional node that went offline (e.g., the Adama edge node) will automatically sync its local WAL changes with the Addis Ababa hub and Hawassa node once network connectivity is restored, closing any data gap that accumulated during the outage.

Conclusion:

The design and architectural planning of this advanced database system successfully provides a highly scalable, secure, and distributed solution optimized for the modern Ethiopian E-Commerce ecosystem. By integrating advanced database management system (DBMS) principles, this project transitions online retail systems from traditional localized operations into a multi-regional, high-throughput environment.

Through strict adherence to relational database design rules, the system moves from unnormalized structures into a clean, fully 3NF schema that eliminates data redundancies and anomalies. Operational issues like concurrent user conflict and product overselling are resolved by combining strict ACID transaction controls with target isolation levels and row locks. These protections allow the system to handle heavy user traffic without risking database corruption or inventory errors. Furthermore, the system accommodates localized user needs while optimizing network performance through horizontal sharding and a hybrid replication strategy split across Addis Ababa, Adama, and Hawassa.

Security and reliability are maintained through multi-layered protections. Features like Role-Based Access Control, AES-256 data encryption, one-way password hashing, and real-time fraud monitoring via an active audit log defend customer and financial information against unauthorized access. Finally, the database ensures system availability and fault tolerance through Write-Ahead Logging (WAL) and the three-phase ARIES crash recovery algorithm, allowing the platform to maintain data consistency even after unexpected hardware or network failures.

In summary, this project provides a technically sound blueprint for scalable enterprise architectures. It demonstrates how advanced database engineering can directly support digital financial integrations, like Telebirr, and handle distributed workloads while maintaining top performance, security, and reliability.

References:

- ✓ **USE WORKBANCH MYSQL APPLICATION.**
- ✓ **Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019).** *Database System Concepts* (7th ed.). McGraw-Hill Education.
Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems* (4th ed.). Springer.
Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.